

EXPERIMENT – 10

Name: K ANUSRI

Roll No: 2520030432

Title

Inode, Hard Link, Symbolic Link and Memory Mapping using mmap()

Aim

To identify and explain inode information, create and examine hard and symbolic links, and demonstrate memory mapping using mmap().

Theory

An inode stores information about a file, including its inode number and file metadata. A hard link creates another directory entry referring to the same inode as the original file. A symbolic link refers to the path of another file. mmap() maps a file into the process memory address space, allowing the file contents to be accessed through memory.

Algorithm

1. Start the program.
2. Create the original file.
3. Write data into the original file.
4. Create a hard link to the original file.
5. Create a symbolic link to the original file.
6. Obtain the inode information of the original file and hard link.
7. Obtain the inode information of the symbolic link.
8. Display the inode numbers.
9. Open the original file for reading.
10. Map the file into memory using mmap().
11. Display the mapped file contents.
12. Unmap the memory using munmap().
13. Close the file.
14. Stop the program.

Program Code

```
k_anusri@DESKTOP-TM20BAQ: ~$ #include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <string.h>

int main()
{
    const char *original = "exp10_original.txt";
    const char *hardlink = "exp10_hardlink.txt";
    const char *symlink_name = "exp10_symlink.txt";

    unlink(original);
    unlink(hardlink);
    unlink(symlink_name);

    int fd = open(original, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    const char *text = "Experiment 10 demonstrates inode, hard link, symbolic link and mmap.\n";
    write(fd, text, strlen(text));
    close(fd);

    if (link(original, hardlink) < 0)
    {
        perror("link");
        return 1;
    }

    if (symlink(original, symlink_name) < 0)
    {
        perror("symlink");
        return 1;
    }
}
```

```

k_anusri@DESKTOP-TM20BAK  ×  +  ▾
}

struct stat st_original;
struct stat st_hard;
struct stat st_sym;

stat(original, &st_original);
stat(hardlink, &st_hard);
lstat(symlink_name, &st_sym);

printf("Original file inode : %lu\n", (unsigned long)st_original.st_ino);
printf("Hard link inode      : %lu\n", (unsigned long)st_hard.st_ino);
printf("Symbolic link inode : %lu\n", (unsigned long)st_sym.st_ino);

fd = open(original, O_RDONLY);

if (fd < 0)
{
    perror("open");
    return 1;
}

size_t size = st_original.st_size;

char *mapped = mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);

if (mapped == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

printf("\nContent using mmap():\n");
write(STDOUT_FILENO, mapped, size);

munmap(mapped, size);
close(fd);

return 0;
}

```

Execution and Screenshots

Screenshot 1 – Program Execution

```

admin@heeraj@DESKTOP-J39GHP7:~$ gcc exp10_links_mmap.c -o exp10_links_mmap
admin@heeraj@DESKTOP-J39GHP7:~$ ./exp10_links_mmap
Original file inode : 18124
Hard link inode      : 18124
Symbolic link inode : 18125

Content using mmap():
Experiment 10 demonstrates inode, hard link, symbolic link and mmap.

```

Screenshot 2 – Inode and Link Verification

```
admin@heeraaj@DESKTOP-J39GHP7:~$ ls -li exp10_original.txt exp10_hardlink.txt exp10_symlink.txt
18124 -rw-r--r-- 2 admin@heeraaj admin@heeraaj 69 Oct  1 05:52 exp10_hardlink.txt
18124 -rw-r--r-- 2 admin@heeraaj admin@heeraaj 69 Oct  1 05:52 exp10_original.txt
18125 lrwxrwxrwx 1 admin@heeraaj admin@heeraaj 18 Oct  1 05:52 exp10_symlink.txt -> exp10_original.txt
admin@heeraaj@DESKTOP-J39GHP7:~$ cat exp10_symlink.txt
```

Screenshot 3 – Symbolic Link Verification

```
admin@heeraaj@DESKTOP-J39GHP7:~$ cat exp10_symlink.txt
Experiment 10 demonstrates inode, hard link, symbolic link and mmap.
admin@heeraaj@DESKTOP-J39GHP7:~$ |
```

Result

The program successfully demonstrated inode information, hard links, symbolic links, and memory mapping using `mmap()`. The inode numbers and file-link relationships were observed, and the file contents were accessed through memory mapping.